



ÉCOLE POLYTECHNIQUE DE SOUSSE

TP 2 : KERBEROS

Configuration Kerberos Cross-Platform

Élèves :
Hichri SAFA

Enseignante :
Cherif MARIEM

- Année Universitaire 2025-2026

Table des matières

1	TP2	4
1.1	Introduction	4
1.2	Objectif du TP	4
1.3	Exercice 1	5
1.3.1	Introduction	5
1.3.2	Installation du serveur Kerberos	5
1.3.3	Création du Realm	5
1.3.4	Configuration du fichier krb5.conf	5
1.3.5	Démarrage des services	6
1.3.6	Création des Principals	8
1.3.7	Réponse à la question	8
1.3.8	Conclusion	9
1.4	Exercice 2	10
1.4.1	Introduction	10
1.4.2	Créer un SPN pour SSH	10
1.4.3	Créer un SPN pour HTTP	10
1.4.4	Vérification des principals	10
1.4.5	Réponse à la question	12
1.4.6	Conclusion	12
1.5	Exercice 3	13
1.5.1	Introduction	13
1.5.2	Première authentification	13
1.5.3	Vérification du ticket (klist et klist -e)	13
1.5.4	Renouvellement du ticket (kinit -R)	14
1.5.5	Destruction du ticket (kdestroy)	14
1.5.6	Ticket de service (kvno)	14
1.5.7	Authentification avec keytab	15
1.5.8	Réponse à la question	15
1.5.9	Conclusion	16
1.6	Exercice 4	17
1.6.1	Introduction	17
1.6.2	Installation et préparation	17
1.6.3	Lancer la capture Kerberos	17
1.6.4	Analyse des paquets	18
1.6.5	Tableau chronologique	21
1.6.6	Réponse à la question	22
1.6.7	Conclusion	23

Table des figures

1.1	Contenu de fichier <code>/etc/krb5.conf</code>	6
1.2	Sortie de <code>systemctl status krb5-kdc</code> montrant le service actif	7
1.3	Sortie de <code>systemctl status krb5-admin-server</code> montrant le service actif . . .	7
1.4	Résultat de <code>listprincs</code> dans <code>kadmin.local</code> montrant tous les principaux créés	8
1.5	Sortie de <code>klist -k /etc/krb5.keytab</code>	10
1.6	Sortie de <code>klist -k /etc/http.keytab</code> montrant le SPN <code>http</code>	10
1.7	Liste de tous les principaux avec <code>listprincs</code> incluant les SPN	11
1.8	Liste complète des principaux incluant les SPN	12
1.9	Sortie de <code>kalist -e</code> après authentification montrant le TGT avec détails de chiffrement	14
1.10	Sortie de <code>kdestroy</code> puis <code>klist</code> montrant l'absence de tickets	14
1.11	Sortie de <code>klist</code> après <code>kvno</code> , montrant TGT + ticket de service	15
1.12	Authentification réussie avec <code>keytab</code> utilisateur	15
1.13	Vue d'ensemble Wireshark avec filtre Kerberos appliqué	17
1.14	Détails du paquet <code>AS-REQ</code>	18
1.15	Détails du paquet <code>AS-REP</code> montrant le TGT chiffré	19
1.16	Détails du paquet <code>TGS-REQ</code>	20
1.17	Détails du paquet <code>TGS-REP</code> avec le ticket de service	21
1.18	Capture de tous les échanges chronologiquement	21

Liste des tableaux

1.1	Comparaison entre principal utilisateur et principal administrateur	8
1.2	Comparaison entre principal utilisateur et principal service	12
1.3	différences entre un TGT et un ticket de service.	16
1.4	Ordre logique des échanges Kerberos	22

Chapitre 1

TP2

1.1 Introduction

À l'ère du numérique, l'identité est devenue la clé d'accès au monde digital. Chaque connexion, chaque service, chaque application repose sur une question fondamentale :

“Qui es-tu ?”

Dans un environnement réseau non sécurisé, transmettre un mot de passe en clair serait une faiblesse critique. C'est ici qu'intervient Kerberos, un protocole d'authentification conçu pour transformer la confiance en un mécanisme cryptographique robuste basé sur des tickets.

Développé au *Massachusetts Institute of Technology* dans les années 1980, Kerberos s'est imposé comme un standard incontournable dans les infrastructures modernes. Il est aujourd'hui au cœur des environnements d'entreprise, notamment via son intégration avec *Microsoft Active Directory*.

Contrairement aux mécanismes d'authentification traditionnels, Kerberos ne transmet jamais le mot de passe sur le réseau. Il repose sur un système de **tickets chiffrés**, délivrés par un centre de distribution de clés (KDC), garantissant :

- Une authentification mutuelle
- Une sécurité renforcée
- Une performance optimisée
- Une compatibilité multi-plateforme (Linux / Windows)

1.2 Objectif du TP

L'objectif de ce travail pratique est de comprendre en profondeur l'architecture et le fonctionnement du protocole Kerberos à travers une mise en œuvre concrète. Dans ce TP, nous avons configuré un serveur Kerberos sous Linux, créé différents principaux (utilisateurs et services), généré des Service Principal Names (SPN), manipulé les tickets d'authentification via les outils clients (kinit, klist, kvno) et analysé les échanges réseau à l'aide de Wireshark.

Ce TP nous a permis de :

- Mettre en place un environnement d'authentification sécurisé ;
- Comprendre le rôle du KDC, des realms et des principaux ;
- Observer les différentes étapes du processus d'authentification (AS-REQ, AS-REP, TGS-REQ, TGS-REP) ;
- Comparer les mécanismes Kerberos et NTLM ;
- Analyser le trafic réseau afin d'identifier les éléments transmis en clair et ceux protégés par chiffrement.

1.3 Exercice 1

1.3.1 Introduction

Dans cet exercice, j'ai mis en place un serveur d'authentification basé sur le protocole Kerberos. L'objectif était de configurer le serveur KDC (Key Distribution Center), de définir un realm ETUDIANT.LAB, puis de créer différents principaux (administrateur et utilisateurs). Cette configuration permet de comprendre le fonctionnement du système d'authentification Kerberos basé sur les tickets et la gestion centralisée des identités.

1.3.2 Installation du serveur Kerberos

Dans cette première étape, j'ai installé les composants nécessaires au fonctionnement de Kerberos :

1. krb5-kdc : serveur d'authentification (KDC)
2. krb5-admin-server : serveur d'administration
3. krb5-user : outils client (kinit, klist...)

Le KDC (Key Distribution Center) constitue le cœur du système Kerberos. Il est responsable :

- de l'authentification des utilisateurs
- de la délivrance des tickets (TGT et tickets de service)

Cette étape permet de mettre en place l'infrastructure d'authentification.

1.3.3 Création du Realm

Le realm représente le domaine d'authentification Kerberos (équivalent d'un domaine Active Directory).

Avec la commande : **sudo krb5-newrealm** j'ai :

- Créé la base de données Kerberos
- Initialisé le realm ETUDIANT.LAB
- Défini un mot de passe master protégeant la base

Ce mot de passe est critique car il sécurise toute l'infrastructure Kerberos.

1.3.4 Configuration du fichier krb5.conf

Le fichier /etc/krb5.conf contient :

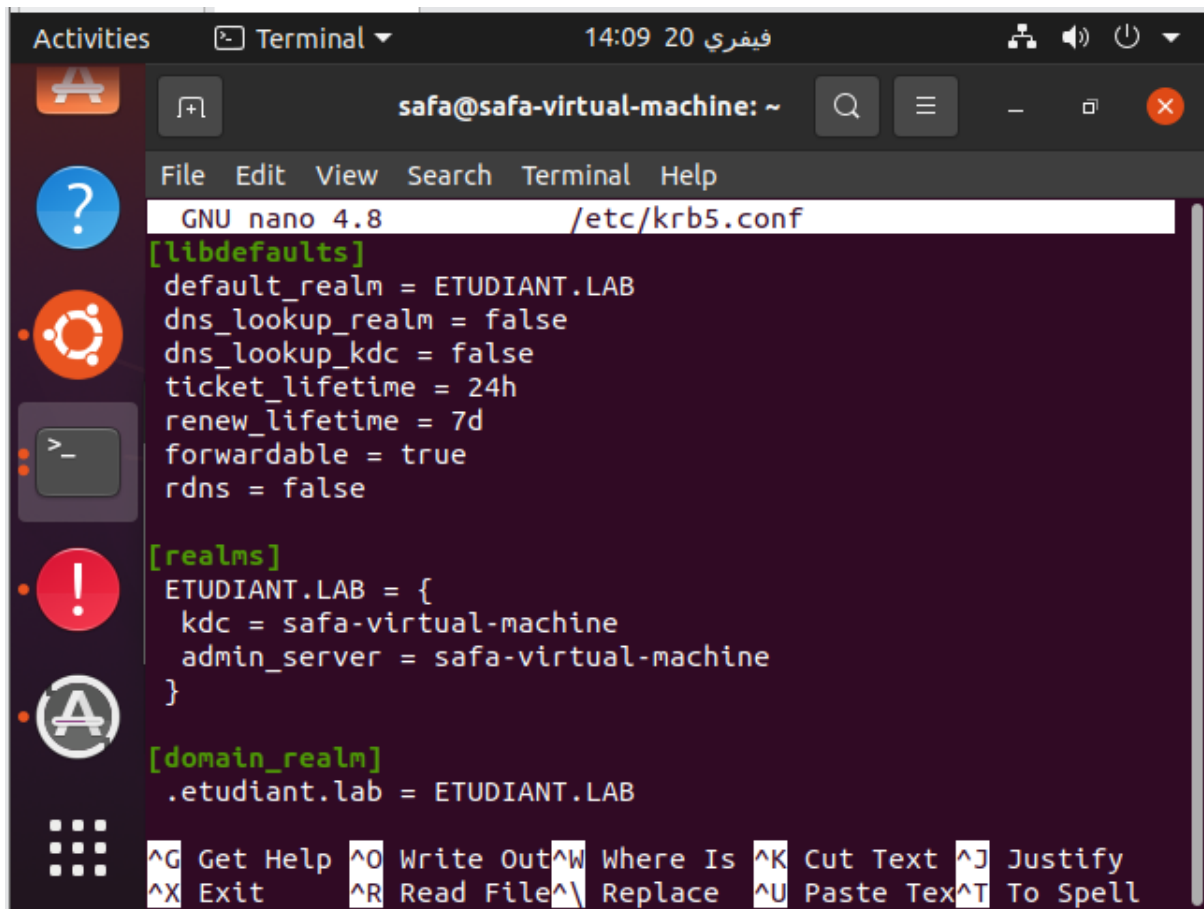
- Le nom du realm par défaut
- L'adresse du KDC
- Les paramètres de sécurité (durée des tickets, renouvellement, etc.)

Il permet au système de savoir :

- Où se trouve le serveur Kerberos
- Comment contacter le KDC

— Quelle politique appliquer aux tickets

Cette configuration est essentielle pour assurer le bon fonctionnement de l'authentification.



```
GNU nano 4.8 /etc/krb5.conf
[libdefaults]
default_realm = ETUDIANT.LAB
dns_lookup_realm = false
dns_lookup_kdc = false
ticket_lifetime = 24h
renew_lifetime = 7d
forwardable = true
rdns = false

[realms]
ETUDIANT.LAB = {
  kdc = safa-virtual-machine
  admin_server = safa-virtual-machine
}

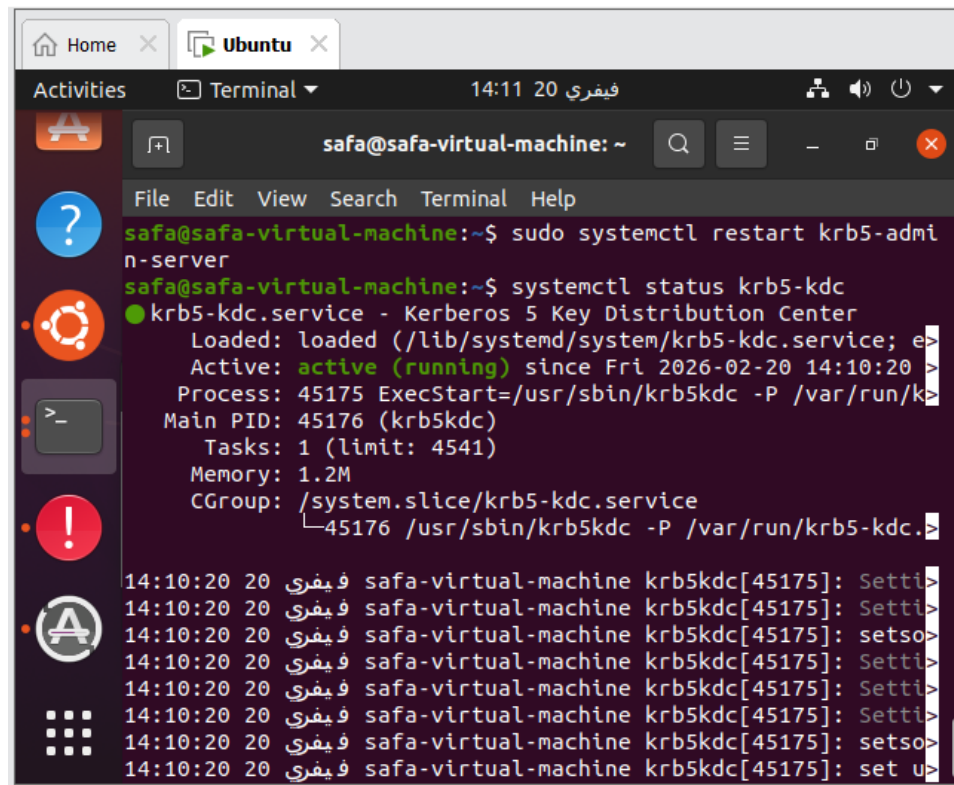
[domain_realm]
.etudiant.lab = ETUDIANT.LAB

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell
```

FIGURE 1.1 – Contenu de fichier `/etc/krb5.conf`

1.3.5 Démarrage des services

Les services `krb5-kdc` et `krb5-admin-server` ont été démarrés puis activés au démarrage du système.



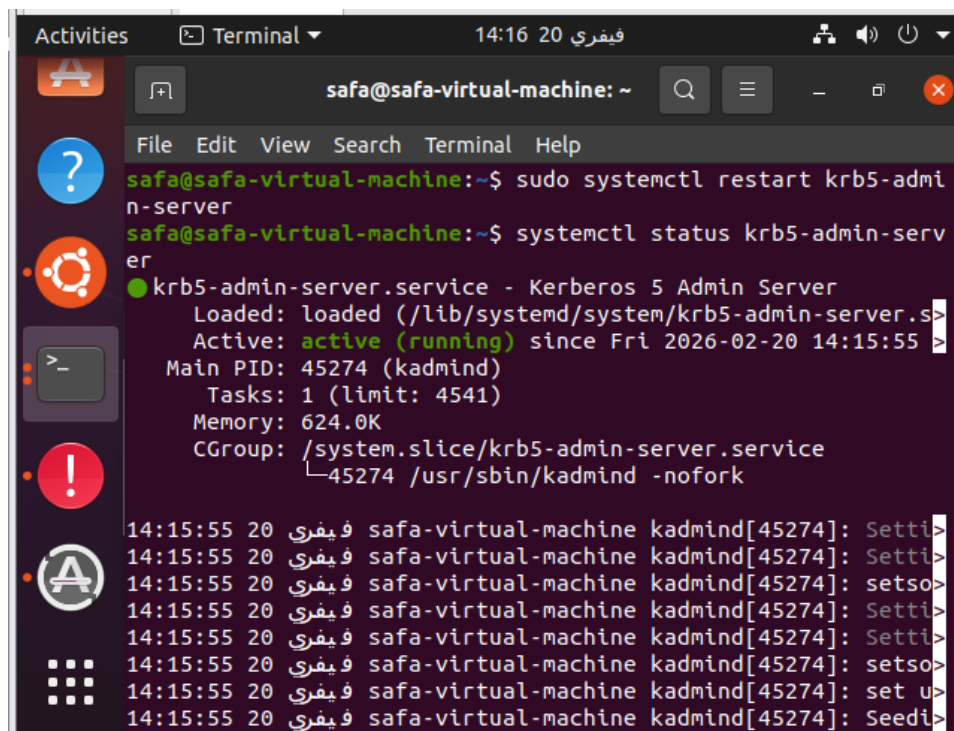
A terminal window titled 'safer@safer-virtual-machine: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The user has executed the following commands:

```
safer@safer-virtual-machine:~$ sudo systemctl restart krb5-admin-server
safer@safer-virtual-machine:~$ systemctl status krb5-kdc
```

The output of the second command shows the status of the `krb5-kdc.service`:

```
● krb5-kdc.service - Kerberos 5 Key Distribution Center
   Loaded: loaded (/lib/systemd/system/krb5-kdc.service; enabled)
   Active: active (running) since Fri 2026-02-20 14:10:20
     Process: 45175 ExecStart=/usr/sbin/krb5kdc -P /var/run/krb5-kdc.pid
    Main PID: 45176 (krb5kdc)
       Tasks: 1 (limit: 4541)
      Memory: 1.2M
     CGroup: /system.slice/krb5-kdc.service
            └─45176 /usr/sbin/krb5kdc -P /var/run/krb5-kdc.pid
```

Below the service status, there is a log of messages from the `krb5kdc` process, showing it is setting up the database and listening for connections.

FIGURE 1.2 – Sortie de `systemctl status krb5-kdc` montrant le service actif

A terminal window titled 'safer@safer-virtual-machine: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The user has executed the following commands:

```
safer@safer-virtual-machine:~$ sudo systemctl restart krb5-admin-server
safer@safer-virtual-machine:~$ systemctl status krb5-admin-server
```

The output of the second command shows the status of the `krb5-admin-server.service`:

```
● krb5-admin-server.service - Kerberos 5 Admin Server
   Loaded: loaded (/lib/systemd/system/krb5-admin-server.service; enabled)
   Active: active (running) since Fri 2026-02-20 14:15:55
     Process: 45274 (kadmind)
       Tasks: 1 (limit: 4541)
      Memory: 624.0K
     CGroup: /system.slice/krb5-admin-server.service
            └─45274 /usr/sbin/kadmind -nofork
```

Below the service status, there is a log of messages from the `kadmind` process, showing it is setting up the database and listening for connections.

FIGURE 1.3 – Sortie de `systemctl status krb5-admin-server` montrant le service actif

1.3.6 Création des Principals

Un principal représente une identité Kerberos (utilisateur ou service).
j'ai créé :

- Un principal administrateur : admin/admin
- Un principal utilisateur : safa@ETUDIANT.LAB
- Un utilisateur de test : testuser@ETUDIANT.LAB

Chaque principal possède :

- Un identifiant unique
- Une clé secrète stockée dans la base Kerberos

La commande listprincs permet d'afficher tous les principals créés.

```
kadmin.local: listprincs
K/M@ETUDIANT.LAB
admin/admin@ETUDIANT.LAB
kadmin/admin@ETUDIANT.LAB
kadmin/changepw@ETUDIANT.LAB
kadmin/safa-virtual-machine@ETUDIANT.LAB
kiprop/safa-virtual-machine@ETUDIANT.LAB
krbtgt/ETUDIANT.LAB@ETUDIANT.LAB
safa@ETUDIANT.LAB
testuser@ETUDIANT.LAB
kadmin.local: quit
safa@safa-virtual-machine:~$
```

FIGURE 1.4 – Résultat de listprincs dans kadmin.local montrant tous les principals créés

1.3.7 Réponse à la question

la différence entre un principal utilisateur et un principal administrateur dans Kerberos.

Critère	Principal utilisateur	Principal administrateur
Définition	Identité utilisée pour s'authentifier et accéder aux services	Identité utilisée pour administrer la base Kerberos
Exemple	safa@ETUDIANT.LAB	admin/admin@ETUDIANT.LAB
Commande utilisée	kinit	kadmin / kadmin.local
Rôle principal	Obtenir un TGT et des tickets de service	Créer, modifier et supprimer des principals
Accès aux services	Oui (SSH, HTTP, etc.)	Oui, mais avec privilèges d'administration
Privilèges	Limités	Élevés
Gestion des comptes	Non	Oui

TABLE 1.1 – Comparaison entre principal utilisateur et principal administrateur

1.3.8 Conclusion

À travers cet exercice, j'ai configuré avec succès un serveur Kerberos et créé plusieurs principaux, dont un administrateur et des utilisateurs standards. j'ai également vérifié le bon fonctionnement des services `krb5-kdc` et `krb5-admin-server`.

1.4 Exercice 2

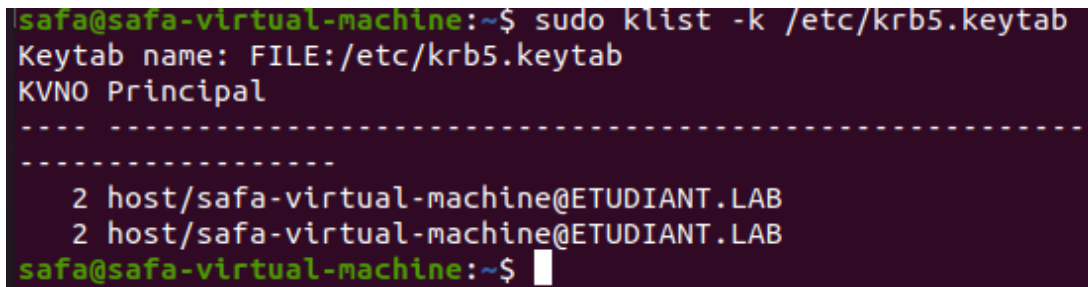
1.4.1 Introduction

Dans cet exercice, nous avons étudié la notion de Service Principal Name (SPN) dans Kerberos. Un SPN représente l'identité d'un service réseau (SSH, HTTP, LDAP) et permet au serveur Kerberos de délivrer des tickets d'authentification adaptés aux services.

1.4.2 Créer un SPN pour SSH

Un principal de type service a été créé pour le service SSH sous la forme : `host/safa-virtual-machine@ETUDIANT.LAB`. L'option `-randkey` a été utilisée afin de générer automatiquement une clé secrète aléatoire.

Un fichier keytab `/etc/krb5.keytab` a ensuite été généré pour permettre au service SSH de s'authentifier automatiquement sans mot de passe.

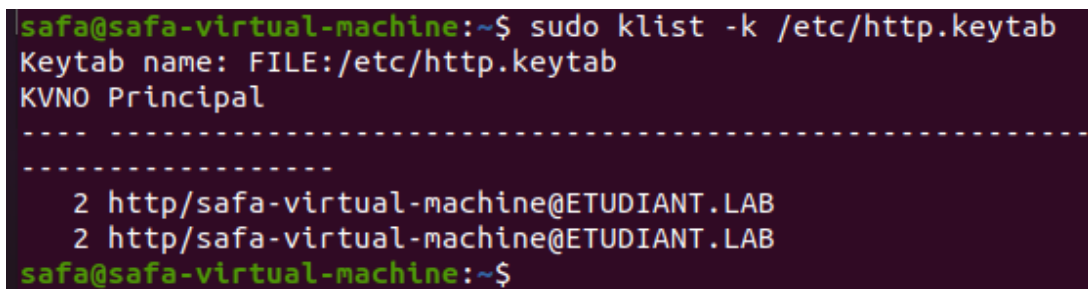


```
safa@safa-virtual-machine:~$ sudo klist -k /etc/krb5.keytab
Keytab name: FILE:/etc/krb5.keytab
KVNO Principal
-----
2 host/safa-virtual-machine@ETUDIANT.LAB
2 host/safa-virtual-machine@ETUDIANT.LAB
safa@safa-virtual-machine:~$
```

FIGURE 1.5 – Sortie de `klist -k /etc/krb5.keytab`

1.4.3 Créer un SPN pour HTTP

Un second SPN a été créé pour le service HTTP : `http/safa-virtual-machine@ETUDIANT.LAB`. Un fichier `/etc/http.keytab` a été généré afin de stocker la clé du service HTTP.



```
safa@safa-virtual-machine:~$ sudo klist -k /etc/http.keytab
Keytab name: FILE:/etc/http.keytab
KVNO Principal
-----
2 http/safa-virtual-machine@ETUDIANT.LAB
2 http/safa-virtual-machine@ETUDIANT.LAB
safa@safa-virtual-machine:~$
```

FIGURE 1.6 – Sortie de `klist -k /etc/http.keytab` montrant le SPN `http`

1.4.4 Vérification des principaux

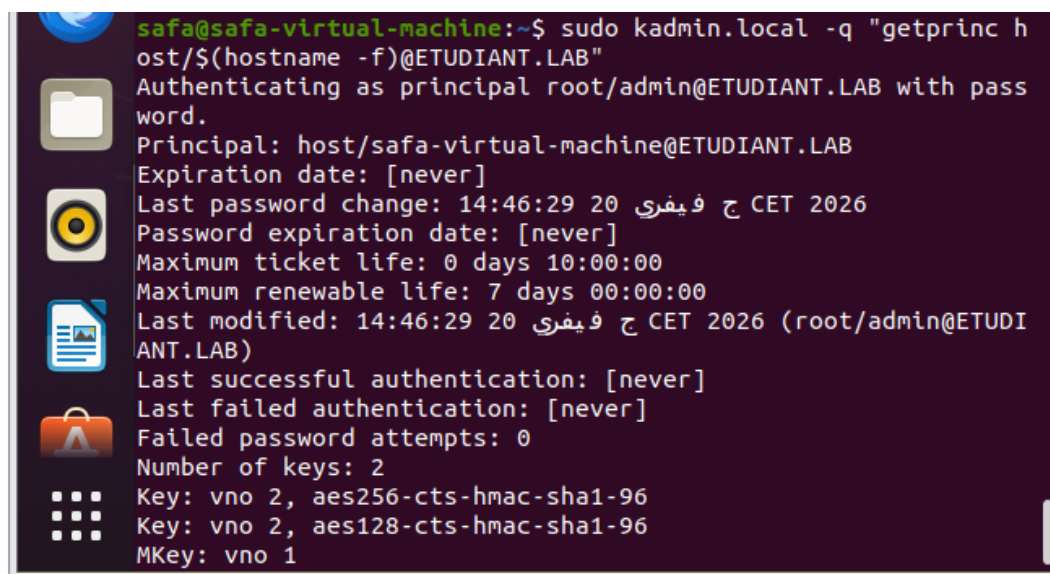
Vérification détaillée d'un principal avec `getprinc`

Après la création des principaux de service (SPN), une vérification a été effectuée afin de confirmer leur bonne configuration dans la base Kerberos du domaine `ETUDIANT.LAB`.

Cette commande permet d'afficher les informations détaillées d'un principal spécifique, notamment :

- Nom complet du principal
- Date de création
- Date d'expiration
- Clés associées
- Politique appliquée
- Type d'authentification

Cette vérification confirme que le principal `host/safa-virtual-machine@ETUDIANT.LAB` est bien enregistré dans la base Kerberos.



```
safa@safa-virtual-machine:~$ sudo kadmin.local -q "getprinc host/$(hostname -f)@ETUDIANT.LAB"
Authenticating as principal root/admin@ETUDIANT.LAB with password.
Principal: host/safa-virtual-machine@ETUDIANT.LAB
Expiration date: [never]
Last password change: 14:46:29 20 ج ويفري CET 2026
Password expiration date: [never]
Maximum ticket life: 0 days 10:00:00
Maximum renewable life: 7 days 00:00:00
Last modified: 14:46:29 20 ج ويفري CET 2026 (root/admin@ETUDIANT.LAB)
Last successful authentication: [never]
Last failed authentication: [never]
Failed password attempts: 0
Number of keys: 2
Key: vno 2, aes256-cts-hmac-sha1-96
Key: vno 2, aes128-cts-hmac-sha1-96
MKey: vno 1
```

FIGURE 1.7 – Liste de tous les principals avec `listprincs` incluant les SPN

Vérification globale avec `listprincs`

La commande **`listprincs`** permet d'afficher la liste complète des principals enregistrés dans la base Kerberos. Cette commande est importante car elle permet de :

- Vérifier que les SPN ont bien été créés
- Confirmer que le KDC reconnaît les services
- S'assurer qu'aucune erreur de nommage n'est présente

```
safa@safa-virtual-machine:~$ sudo kadmin.local -q "listprincs"
"
Authenticating as principal root/admin@ETUDIANT.LAB with pass
word.
K/M@ETUDIANT.LAB
admin/admin@ETUDIANT.LAB
host/safa-virtual-machine@ETUDIANT.LAB
http/safa-virtual-machine@ETUDIANT.LAB
kadmin/admin@ETUDIANT.LAB
kadmin/changepw@ETUDIANT.LAB
kadmin/safa-virtual-machine@ETUDIANT.LAB
kiprop/safa-virtual-machine@ETUDIANT.LAB
krbtgt/ETUDIANT.LAB@ETUDIANT.LAB
safa@ETUDIANT.LAB
testuser@ETUDIANT.LAB
safa@safa-virtual-machine:~$
```

FIGURE 1.8 – Liste complète des principals incluant les SPN

1.4.5 Réponse à la question

Pourquoi utilise-t-on `-randkey` ?

On utilise `-randkey` pour :

- Générer automatiquement une clé cryptographique forte
- Éviter l'utilisation d'un mot de passe humain
- Sécuriser les comptes de service

Les services n'ont pas besoin de mot de passe humain → ils utilisent une clé stockée dans un keytab.

Différence avec un principal utilisateur ?

Principal utilisateur	Principal service (SPN)
Possède un mot de passe	Utilise une clé aléatoire
Utilisé par une personne	Utilisé par un service
Authentification interactive	Authentification automatique
Ex : safa@ETUDIANT.LAB	Ex : host/server@REALM

TABLE 1.2 – Comparaison entre principal utilisateur et principal service

1.4.6 Conclusion

Cet exercice nous a permis de créer et vérifier les principals Kerberos dans le domaine ETUDIANT.LAB. Grâce aux commandes `getprinc` et `listprincs`, nous avons confirmé que les comptes utilisateurs et les SPN sont correctement enregistrés dans la base Kerberos.

La configuration est donc validée et le système est prêt pour une authentification sécurisée basée sur Kerberos.

1.5 Exercice 3

1.5.1 Introduction

Dans cet exercice, nous avons testé le mécanisme d'authentification Kerberos à l'aide de la commande `kinit`. L'objectif était de comprendre le cycle de vie d'un ticket Kerberos, depuis sa création jusqu'à sa destruction, ainsi que son renouvellement. Nous avons également étudié l'obtention d'un ticket de service et l'authentification automatique à l'aide d'un keytab.

1.5.2 Première authentification

Commande : **kinit safa@ETUDIANT.LAB**

Cette commande permet de s'authentifier auprès du KDC (Key Distribution Center).

Fonctionnement :

1. L'utilisateur envoie une demande d'authentification au KDC.
2. Il prouve son identité en saisissant son mot de passe.
3. Si le mot de passe est correct, le KDC délivre un TGT (Ticket Granting Ticket).
4. Ce TGT sera utilisé pour demander d'autres tickets sans ressaisir le mot de passe.

À ce stade, aucun ticket de service n'est encore obtenu.

1.5.3 Vérification du ticket (`klist` et `klist -e`)

La commande **klist** affiche les tickets présents dans le cache Kerberos.

On observe :

- Le principal par défaut
- Le ticket `krbtgt/ETUDIANT.LAB`
- La date de début
- La date d'expiration

La commande `klist -e` affiche en plus :

- Le type de chiffrement utilisé (ex : AES-256)

Cela permet de vérifier :

- Que le TGT a bien été généré
- Sa durée de validité
- Son niveau de sécurité

```
safa@safa-virtual-machine:~$ klist -e
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: safa@ETUDIANT.LAB

Valid starting          Expires                Service principal
2026-02-20T15:52:04    2026-02-21T01:52:04  krbtgt/ETUDIANT.LAB
@ETUDIANT.LAB
        renew until 2026-02-27T15:51:55, Etype (skey, tkt): a
es256-cts-hmac-sha1-96, aes256-cts-hmac-sha1-96
safa@safa-virtual-machine:~$
```

FIGURE 1.9 – Sortie de `klist -e` après authentification montrant le TGT avec détails de chiffrement

1.5.4 Renouvellement du ticket (`kinit -R`)

Commande : **kinit -R**

Cette commande permet de renouveler le TGT avant son expiration.

Important :

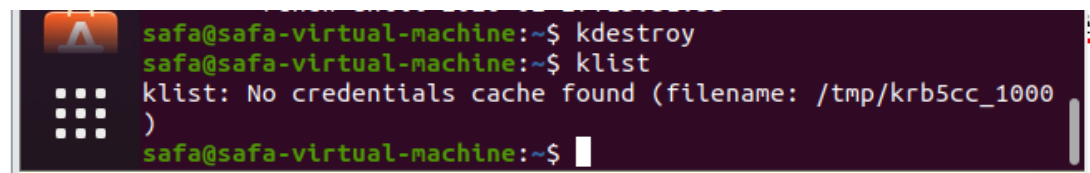
- Le mot de passe n'est pas redemandé.
- La date d'expiration est repoussée.
- La date "renew until" ne change pas.

Cela montre que le ticket possède le flag Renewable (R).

1.5.5 Destruction du ticket (`kdestroy`)

Commande : **kdestroy**

Cette commande supprime tous les tickets présents dans le cache Kerberos



```
safa@safa-virtual-machine:~$ kdestroy
safa@safa-virtual-machine:~$ klist
klist: No credentials cache found (filename: /tmp/krb5cc_1000)
safa@safa-virtual-machine:~$
```

FIGURE 1.10 – Sortie de `kdestroy` puis `klist` montrant l'absence de tickets

1.5.6 Ticket de service (`kvno`)

Cette commande demande un ticket de service spécifique pour le service host.

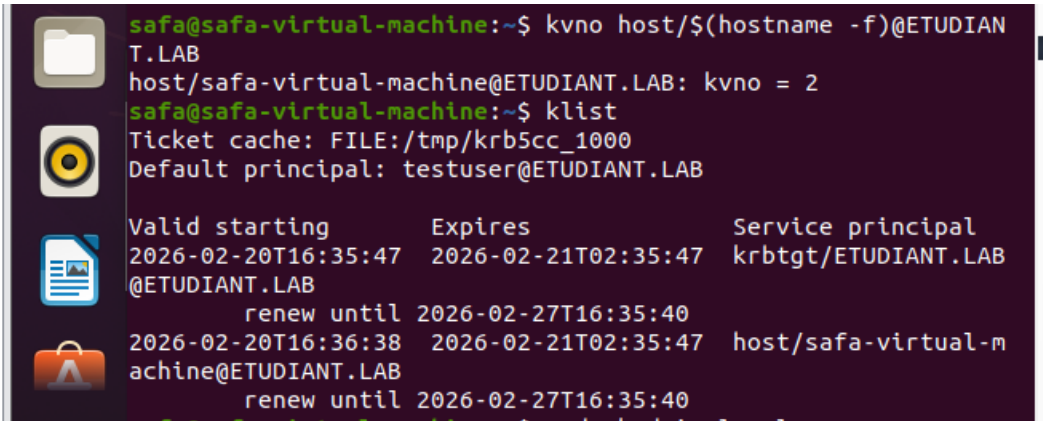
Fonctionnement :

1. L'utilisateur utilise son TGT.
2. Il demande un ticket pour un service précis.
3. Le KDC délivre un ticket de service (TS).

Après `klist`, on observe :

- Le TGT
- Le ticket pour host/...

Maintenant je peut accéder au service sans ressaisir mon mot de passe.



```
safa@safa-virtual-machine:~$ kvno host/$(hostname -f)@ETUDIANT.LAB
host/safa-virtual-machine@ETUDIANT.LAB: kvno = 2
safa@safa-virtual-machine:~$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: testuser@ETUDIANT.LAB

Valid starting      Expires            Service principal
2026-02-20T16:35:47 2026-02-21T02:35:47 krbtgt/ETUDIANT.LAB
@ETUDIANT.LAB
renew until 2026-02-27T16:35:40
2026-02-20T16:36:38 2026-02-21T02:35:47 host/safa-virtual-m
achine@ETUDIANT.LAB
renew until 2026-02-27T16:35:40
```

FIGURE 1.11 – Sortie de klist après kvno, montrant TGT + ticket de service

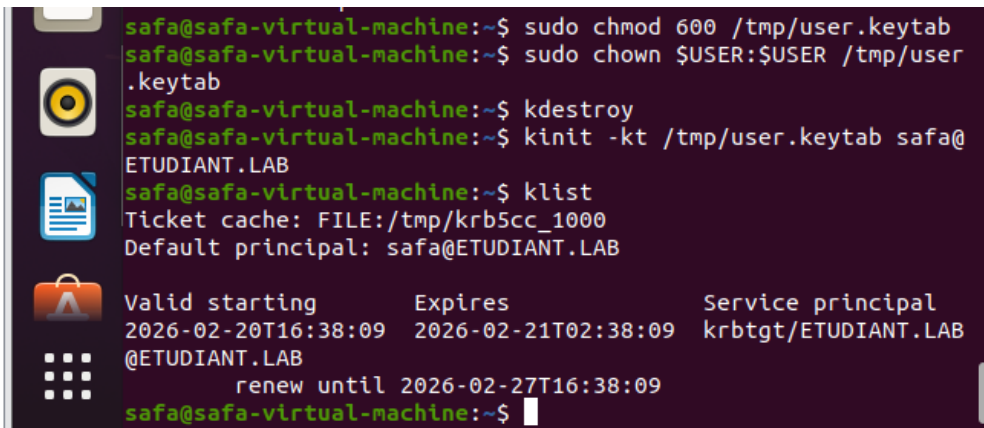
1.5.7 Authentification avec keytab

Un keytab est un fichier contenant une clé secrète associée à un principal.

Utilité :

- Permet une authentification automatique.
- Aucun mot de passe n'est demandé.
- Utilisé principalement pour les services et scripts automatisés.

C'est une méthode adaptée aux environnements serveurs.



```
safa@safa-virtual-machine:~$ sudo chmod 600 /tmp/user.keytab
safa@safa-virtual-machine:~$ sudo chown $USER:$USER /tmp/user
.keytab
safa@safa-virtual-machine:~$ kdestroy
safa@safa-virtual-machine:~$ kinit -kt /tmp/user.keytab safa@
ETUDIANT.LAB
safa@safa-virtual-machine:~$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: safa@ETUDIANT.LAB

Valid starting      Expires            Service principal
2026-02-20T16:38:09 2026-02-21T02:38:09 krbtgt/ETUDIANT.LAB
@ETUDIANT.LAB
renew until 2026-02-27T16:38:09
safa@safa-virtual-machine:~$
```

FIGURE 1.12 – Authentification réussie avec keytab utilisateur

1.5.8 Réponse à la question

Durée de vie par défaut du TGT ?

On regarde dans klist. Généralement :

- 10 heures (0 days 10 :00 :00)
- Renouvelable 7 jours

Différence entre TGT et ticket de service ?

TGT	Ticket de Service
Ticket général	Ticket pour un service précis
Donné par le KDC	Donné après demande avec le TGT
Permet de demander d'autres tickets	Permet d'accéder à SSH, HTTP, etc.

TABLE 1.3 – différences entre un TGT et un ticket de service.

Pourquoi les keytabs sont utiles ?

- Permettent l'authentification automatique
- Pas besoin de mot de passe
- Utilisés pour services (HTTP, SSH, LDAP)
- Idéal pour scripts et serveurs

1.5.9 Conclusion

Cet exercice nous a permis de valider le fonctionnement du processus d'authentification Kerberos. Nous avons observé la création d'un TGT, son renouvellement, l'obtention d'un ticket de service et sa suppression. Le test avec un keytab a montré comment Kerberos peut être utilisé pour une authentification automatisée, essentielle dans les environnements serveurs et services techniques.

Ainsi, le cycle complet de gestion des tickets Kerberos a été compris et vérifié avec succès.

1.6 Exercice 4

1.6.1 Introduction

Dans cet exercice, nous avons analysé le fonctionnement du protocole Kerberos à l'aide de Wireshark. L'objectif était d'observer les échanges entre le client et le KDC, d'identifier les messages AS-REQ, AS-REP, TGS-REQ et TGS-REP, et de comprendre comment Kerberos sécurise l'authentification grâce aux tickets et au chiffrement.

1.6.2 Installation et préparation

Wireshark est un outil d'analyse réseau qui permet d'intercepter et d'examiner les paquets circulant sur le réseau. L'ajout de l'utilisateur au groupe wireshark permet de capturer des paquets sans privilèges root, ce qui améliore la sécurité et la praticité.

Cette étape garantit que nous disposons d'un environnement prêt pour analyser le trafic Kerberos.

1.6.3 Lancer la capture Kerberos

Dans Wireshark :

1. L'interface réseau a été sélectionnée :lo
2. Le filtre kerberos a été appliqué.
3. La capture a été lancée.

À ce stade, Wireshark est prêt à enregistrer tous les échanges Kerberos.

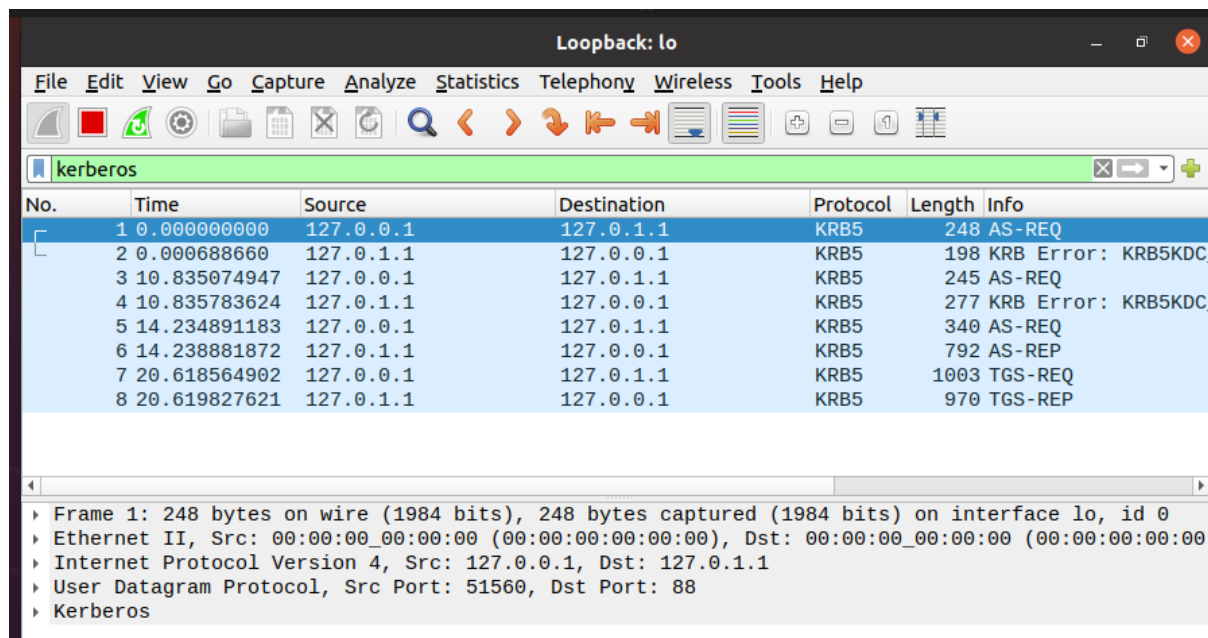


FIGURE 1.13 – Vue d'ensemble Wireshark avec filtre Kerberos appliqué

1.6.4 Analyse des paquets

Analyse de AS-REQ

Le paquet AS-REQ correspond à la première demande envoyée par le client au KDC. Dans ce paquet, nous avons observé :

- cname : safa
- realm : ETUDIANT.LAB
- etype : 18 (AES256)

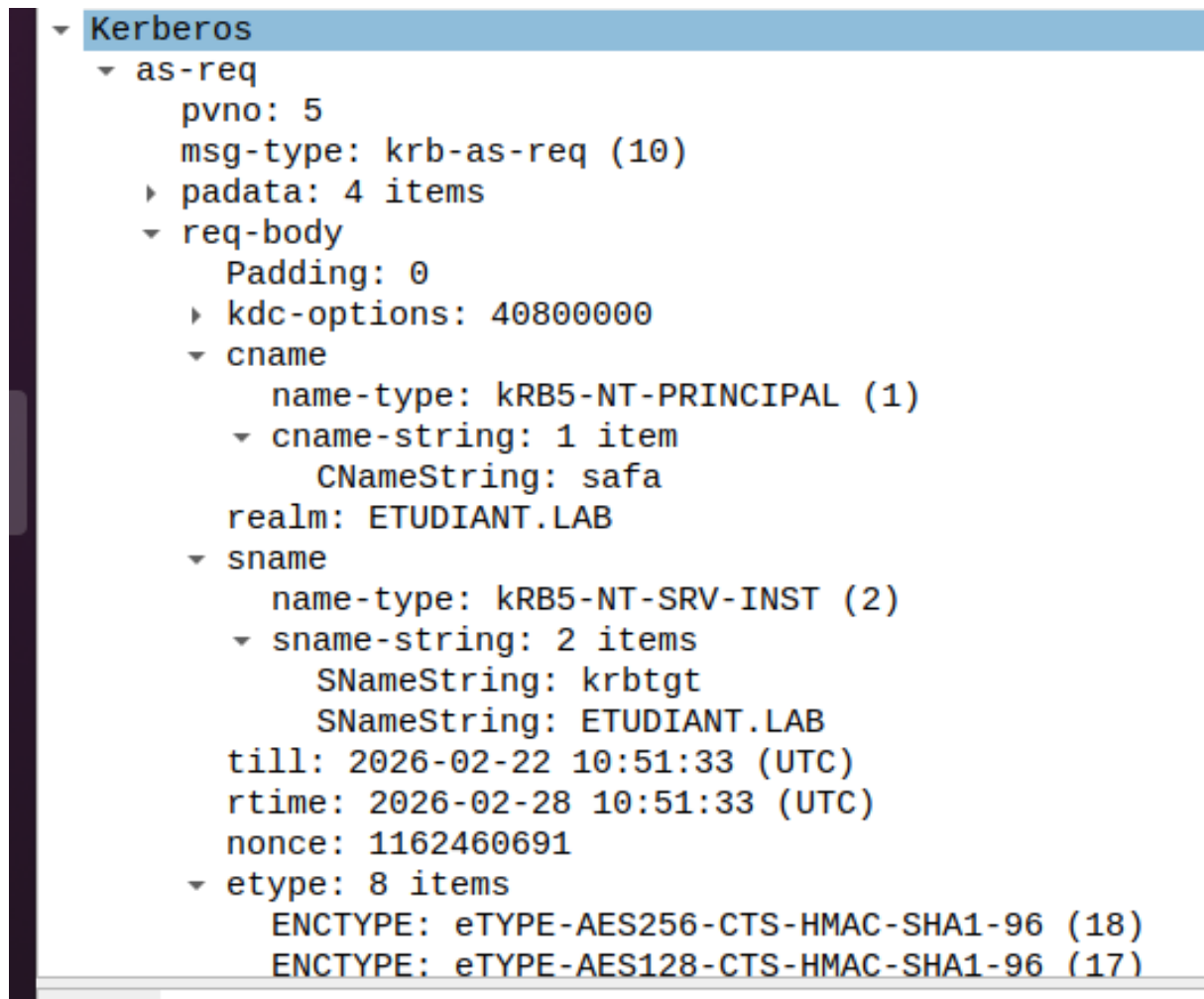


FIGURE 1.14 – Détails du paquet AS-REQ

Analyse :

- Le nom d'utilisateur est transmis en clair.
- Le mot de passe n'est jamais envoyé.
- Le client annonce les algorithmes qu'il supporte.
- Une erreur PREAUTH-REQUIRED est observée, indiquant que la pré-authentification est activée.

Cela démontre que Kerberos protège le mot de passe et exige une preuve d'identité avant d'envoyer un ticket.

Analyse de AS-REP

Le paquet AS-REP est la réponse du KDC à la demande AS-REQ.

Il contient :

- Le TGT (Ticket Granting Ticket).
- Une clé de session.
- Une partie chiffrée (enc-part).

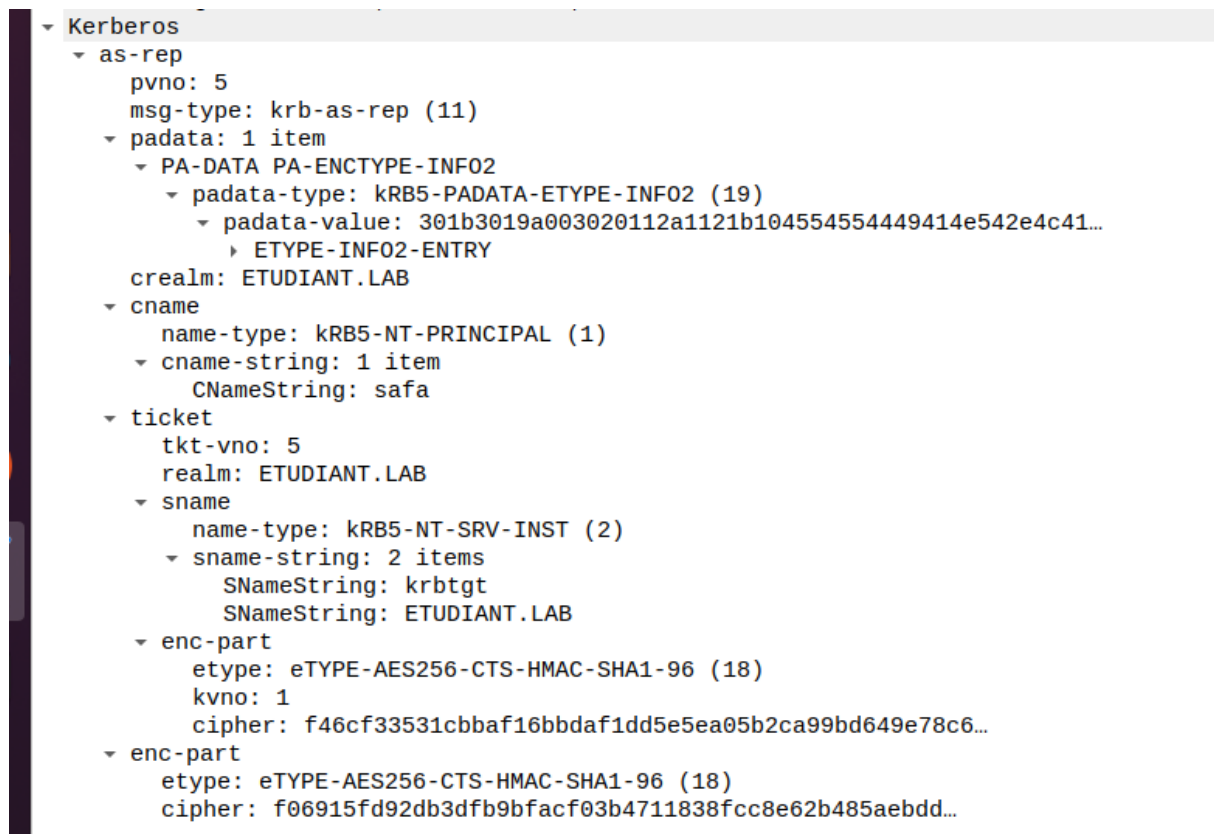


FIGURE 1.15 – Détails du paquet AS-REP montrant le TGT chiffré

Le TGT est chiffré avec la clé du TGS, ce qui signifie que le client ne peut pas le lire. Il peut uniquement le transmettre au TGS ultérieurement. L'algorithme observé est :

- etype 18 → AES256-CTS-HMAC-SHA1-96

Ce chiffrement est considéré comme robuste.

Analyse de TGS-REQ

Une fois authentifié, le client demande un ticket de service. Dans TGS-REQ, nous observons :

- Le TGT inclus dans la requête.
- Le nom du service demandé : host/safa-virtual-machine
- Un authenticator chiffré.

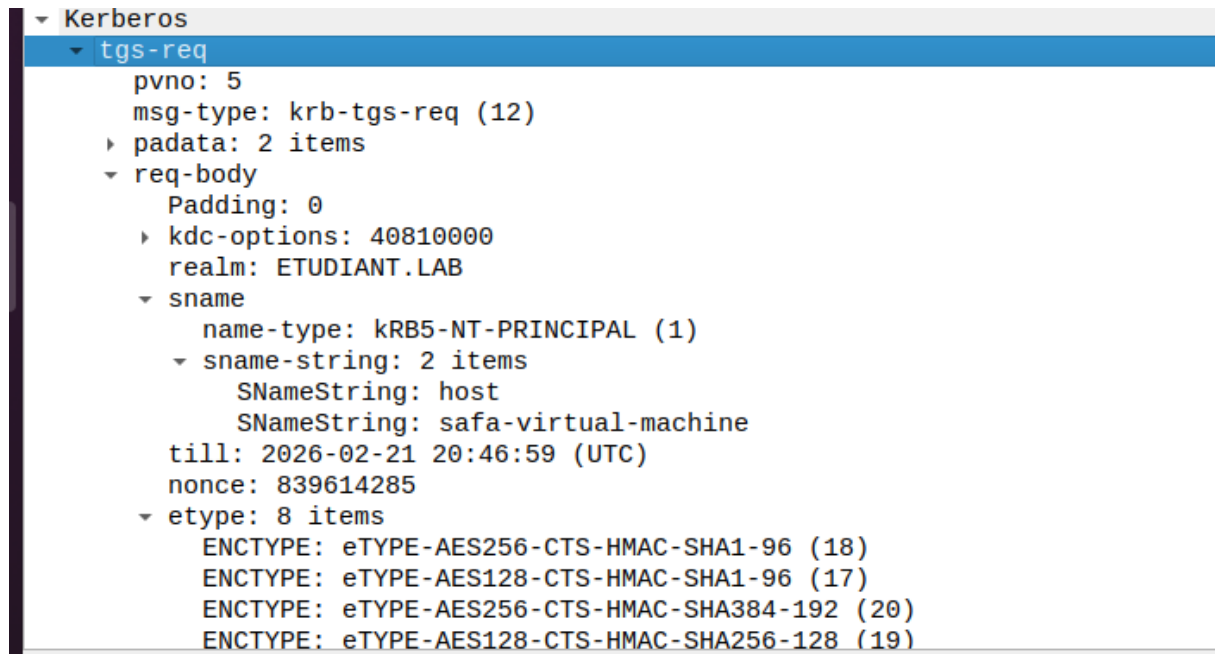


FIGURE 1.16 – Détails du paquet TGS-REQ

Cela signifie que le client prouve qu'il possède la clé de session obtenue précédemment.

Analyse de TGS-REP

Le TGS-REP contient :

- Le ticket de service.
- Une nouvelle clé de session.
- Une partie chiffrée



FIGURE 1.17 – Détails du paquet TGS-REP avec le ticket de service

Ce ticket est destiné uniquement au service demandé. Le client l'utilisera pour accéder au service sans retransmettre son mot de passe.

1.6.5 Tableau chronologique

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	KRB5	1003	TGS-REQ
2	0.001070396	127.0.0.1	127.0.0.1	KRB5	970	TGS-REP
3	188.240192188	127.0.0.1	127.0.0.1	KRB5	245	AS-REQ
4	188.241068242	127.0.0.1	127.0.0.1	KRB5	277	KRB Error: KRB5KDC_ERR_PREAUTH_REQUIRED
5	192.456369057	127.0.0.1	127.0.0.1	KRB5	340	AS-REQ
6	192.460139724	127.0.0.1	127.0.0.1	KRB5	792	AS-REP

FIGURE 1.18 – Capture de tous les échanges chronologiquement

Ordre logique	Type	Source	Destination	Temps	Description
1	AS-REQ	127.0.0.1	127.0.1.1	188.240	Demande TGT
2	AS-REP	127.0.1.1	127.0.0.1	192.460	Envoi TGT
3	TGS-REQ	127.0.0.1	127.0.1.1	0.000	Demande ticket service
4	TGS-REP	127.0.1.1	127.0.0.1	0.001	Envoi ticket service

TABLE 1.4 – Ordre logique des échanges Kerberos

1.6.6 Réponse à la question

algorithme de chiffrement a été utilisé pour votre TGT ?

D'après l'analyse du paquet AS-REP (msg-type = 11) dans Wireshark, le champ etype indique la valeur 18, correspondant à l'algorithme : **AES256-CTS-HMAC-SHA1-96**. Cela signifie que le Ticket Granting Ticket (TGT) est chiffré avec un algorithme symétrique fort basé sur AES en 256 bits.

Ce choix garantit un haut niveau de sécurité et correspond aux configurations modernes de Kerberos.

Pourquoi le mot de passe n'apparaît-il jamais dans les paquets capturés ?

Le mot de passe n'est jamais transmis sur le réseau car Kerberos repose sur un mécanisme cryptographique sécurisé.

Quelle est la différence observable entre AS-REP et TGS-REP ?

La différence principale concerne le type de ticket transmis et l'objectif du message.

AS-REP :

- Réponse du Authentication Service (AS).
- Contient le Ticket Granting Ticket (TGT).
- Sert à prouver l'identité de l'utilisateur pour les demandes futures.

TGS-REP :

- Réponse du Ticket Granting Service (TGS).
- Contient un ticket de service spécifique.
- Permet d'accéder à un service particulier

Combien de temps s'est écoulé entre AS-REQ et TGS-REP ?

$192.460 - 188.240 = 4.22$ secondes environ

Il s'est écoulé environ 4,22 secondes entre l'AS-REQ et l'AS-REP. Le ticket de service est délivré presque instantanément après la demande TGS (0,001 seconde).

Identifiez au moins 3 éléments transmis en clair dans les paquets Kerberos

- Le nom d'utilisateur (cname)
- Le realm
- Le nom du service demandé
- Les types de chiffrement supportés (etype)
- Les adresses IP source et destination

Ces informations ne compromettent pas la sécurité du mot de passe, mais elles permettent d'identifier le contexte de l'authentification.

1.6.7 Conclusion

Cette analyse nous a permis de visualiser le processus complet d'authentification Kerberos. Nous avons constaté que le mot de passe n'est jamais transmis sur le réseau et que les tickets sont protégés par un chiffrement fort. Cet exercice a permis de relier la théorie du protocole à son fonctionnement réel sur le réseau.

